

**МИНИСТЕРСТВО ЦИФРОВОГО РАЗВИТИЯ, СВЯЗИ И МАССОВЫХ
КОММУНИКАЦИЙ РОССИЙСКОЙ ФЕДЕРАЦИИ**

**Ордена трудового Красного Знамени федеральное государственное
бюджетное**

**образовательное учреждение высшего образования
«Московский технический университет связи и информатики»**

Кафедра Математическая кибернетика и информационные технологии

Лабораторная работа №7

Многопоточность

по дисциплине

«Информационные технологии и программирование»

Выполнил: студент гр. БВТ2403
Толибов Р.Ш.

Руководитель:

Москва, 2025 г.

Цель работы: изучить принципы многопоточности в языке Java, освоить создание и управление потоками с использованием классов Thread, Runnable и библиотеки Executors, а также применить механизмы синхронизации для безопасного доступа к общим ресурсам.

Ход работы:

Многопоточность в Java позволяет выполнять несколько задач параллельно, используя несколько потоков выполнения, что повышает производительность приложений. Поток (Thread) — это отдельный путь выполнения программы. Для безопасной работы с общими ресурсами используются механизмы синхронизации: ключевые слова synchronized и volatile, а также объекты блокировок (Lock).

Основные способы создания потоков:

1. Расширение класса Thread и переопределение метода run().
2. Реализация интерфейса Runnable и передача его экземпляра в конструктор Thread.
3. Использование библиотеки Executors, которая позволяет создавать пулы потоков и управлять планировкой задач.

Ключевое слово synchronized обеспечивает эксклюзивный доступ к критической секции кода, предотвращая состояния гонки. Ключевое слово volatile гарантирует видимость изменений переменной всеми потоками, что особенно важно для флагов и индикаторов состояния.

Задание 1. Многопоточное вычисление суммы элементов массива.

Была реализована программа, которая делит массив на две половины, каждая из которых обрабатывается отдельным потоком. После завершения работы потоков результаты складываются в главном потоке.

Также был реализован вариант с использованием класса Thread, где массив делится на равные части, и каждая часть обрабатывается отдельной задачей. Результаты суммирования объединяются после завершения всех потоков.

```
public class MultiThreadSum {  
  
    static class SumThread extends Thread {  
        private int[] array;  
        private int start;  
        private int end;  
        private int result = 0;  
  
        public SumThread(int[] array, int start, int end) {  
            this.array = array;  
            this.start = start;  
            this.end = end;  
        }  
    }  
}
```

```

        @Override
        public void run() {
            for (int i = start; i < end; i++) {
                result += array[i];
            }
        }
        public int getResult() {
            return result;
        }
    }

    public static void main(String[] args) throws InterruptedException {
        int[] data = {1, 2, 3, 4, 5, 6, 7, 8};

        int mid = data.length / 2;

        SumThread t1 = new SumThread(data, 0, mid);
        SumThread t2 = new SumThread(data, mid, data.length);

        t1.start();
        t2.start();

        t1.join();
        t2.join();

        System.out.println("t1 = " + t1.getResult());
        System.out.println("t2 = " + t2.getResult());

        int totalSum = t1.getResult() + t2.getResult();

        System.out.println("t1 + t2 = " + totalSum);
    }
}

```

Задание 2. Многопоточный поиск наибольшего элемента в матрице

Вариант с созданием потоков: каждая строка матрицы обрабатывается отдельным потоком, который находит максимальный элемент в своей строке. Главный поток собирает результаты и определяет наибольший элемент среди всех строк.

Матрица делится на блоки строк, каждый блок обрабатывается отдельной задачей в пуле потоков. После завершения всех потоков результаты сравниваются для нахождения максимального элемента.

```
public class MultiThreadSum {

    static class SumThread extends Thread {
        private int[] array;
        private int start;
        private int end;
        private int result = 0;
        public SumThread(int[] array, int start, int end) {
            this.array = array;
            this.start = start;
            this.end = end;
        }
        @Override
        public void run() {
            for (int i = start; i < end; i++) {
                result += array[i];
            }
        }
        public int getResult() {
            return result;
        }
    }

    public static void main(String[] args) throws InterruptedException {
        int[] data = {1, 2, 3, 4, 5, 6, 7, 8};
        int mid = data.length / 2;
        SumThread t1 = new SumThread(data, 0, mid);
        SumThread t2 = new SumThread(data, mid, data.length);
        t1.start();
        t2.start();
        t1.join();
        t2.join();
        System.out.println("t1 = " + t1.getResult());
        System.out.println("t2 = " + t2.getResult());
        int totalSum = t1.getResult() + t2.getResult();
        System.out.println("t1 + t2 = " + totalSum);
    }
}
```

Задание 3. Многопоточная работа грузчиков на складе

Созданы классы Товар, Склад и Грузчик. Каждый грузчик реализован как отдельный поток, который переносит товары с ограничением суммарного веса до 150 кг за одну итерацию. После достижения лимита поток отправляет груз на другой склад и продолжает обработку оставшихся товаров.

Использование потоков позволяет моделировать параллельную работу нескольких грузчиков, а синхронизация обеспечивает корректный доступ к общему списку товаров на складе.

```
public class Товар {
    private final String name;
    private final int weight;

    public Товар(String name, int weight) {
        this.name = name;
        this.weight = weight;
    }

    public int getWeight() {
        return weight;
    }

    public String getName() {
        return name;
    }
}
```

```
import java.util.List;

public class Склад {
    private final List<Товар> товари;
    private int currentWeight = 0;
    private final int MAX_WEIGHT = 150;

    public Склад(List<Товар> товари) {
        this.товари = товари;
    }

    public synchronized Товар takeТовар() {
        if (товари.isEmpty()) {
            return null;
        }
        return товари.remove(0);
    }

    public synchronized boolean addWeight(int weight) {
        if (currentWeight + weight <= MAX_WEIGHT) {
```

```

        currentWeight += weight;
        return true;
    }
    return false;
}

public synchronized void sendPartiya() {
    System.out.println("Грузчики уносят партию весом: " +
currentWeight + " кг\n");
    currentWeight = 0;
}

public synchronized boolean isEmpty() {
    return tovari.isEmpty();
}

public synchronized int getCurrentWeight() {
    return currentWeight;
}

public synchronized void process(Gruzchik g, Tovar t) {
    int w = t.getWeight();

    if (!addWeight(w)) {
        sendPartiya();
        addWeight(w);
    }

    System.out.println(g.getName() + " взял товар '" + t.getName()
+ "' весом " +
    t.getWeight() + " кг (текущий вес партии: " + currentWeight +
    ")");
}
}

```

```

public class Gruzchik extends Thread {
    private final Sklad sklad;

    public Gruzchik(String name, Sklad sklad) {
        super(name);
        this.sklad = sklad;
    }

    @Override
    public void run() {
        while (true) {
            Tovar t = sklad.takeTovar();
            if (t == null) break;

            sklad.process(this, t);

        }
        try {
            Thread.sleep(200);
        } catch (InterruptedException e) {
            e.printStackTrace();
        }
    }
}

```

```

import java.util.ArrayList;
import java.util.List;

public class Main {
    public static void main(String[] args) throws InterruptedException {

        List<Tovar> list = new ArrayList<>();

        list.add(new Tovar("Бутили воды", 30));
        list.add(new Tovar("Ящик бананов", 20));
        list.add(new Tovar("Мешок сахара", 45));
        list.add(new Tovar("Ящик апельсинов", 35));

        Sklad sklad = new Sklad(list);

        Gruzchik g1 = new Gruzchik("Грузчик 1", sklad);
        Gruzchik g2 = new Gruzchik("Грузчик 2", sklad);
        Gruzchik g3 = new Gruzchik("Грузчик 3", sklad);

        g1.start();
        g2.start();
        g3.start();

        g1.join();
        g2.join();
    }
}

```

```
        g3.join();

        if (sklad.getCurrentWeight() > 0) {
            sklad.sendPartiya();
        }
    }
}
```

Вывод: в результате лабораторной работы были изучены основные концепции многопоточности в Java: создание потоков, управление их жизненным циклом, синхронизация и использование пулов потоков. Многопоточность позволяет повысить эффективность приложений и безопасно обрабатывать общие ресурсы. Полученные знания применимы в разработке высокопроизводительных и параллельных программ.

<https://github.com/TRuslan666/ITiP/tree/main/лаб7>